Quiz 9: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 9 is Thursday Oct 29, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

> ❗ There is no coding section
>
> Quizzes 3 through 7 ended with a "find the bugs in this script" exercise. That is gone for good. Part D is **short answer**: one or two sentences explaining why something behaves the way it does.
> Reading code is a skill this course uses constantly. Debugging syntax is not one it grades.

Scope

The Oct 22 lecture, which was two things: the rest of the sequence models deck, which Oct 15 left at embeddings, and then transformers.

On the quiz:

Finishing sequence data

- padding a ragged batch, and where T comes from
- masking, and the three places it is needed
- `collate_fn`, and bucketing by length
- 1D convolution over a sequence, and its receptive field
- the recurrence, and weight sharing across time
- reading a recurrent layer's output
- autoregressive generation and teacher forcing
- backpropagation through time, and the two failures it produces
- what gating fixed, and what it left alone

Transformers

- **query, key and value**, and what the attention operation computes
- the scaled dot product, and why it is divided by $\sqrt{d_k}$
- the attention weight matrix, and where the quadratic cost comes from
- **multi-head attention**, and what a head is for
- **positional encoding**, sinusoidal and learned
- the **padding mask** and the **causal mask**
- the transformer block: residual, feed forward network, **layer normalization**
- encoder only, decoder only, encoder-decoder, and which models are which

Not on the quiz. Anything from the Oct 29 lecture, which is large language models: scaling laws, RoPE, RMSNorm, the KV cache, SFT, RLHF, DPO, decoding strategies. Also off: deriving the sinusoidal formula from scratch, GRU internals, bidirectional RNNs, and writing an LSTM cell by hand.

Part 1: Padding

A batch has to be a rectangle, because a tensor has a shape. Sentences are not the same length. **Padding appends a reserved token until every sequence reaches the length of the longest one in the batch.**

$$T = \max_i \ell_i, \qquad \text{cells} = N \times T, \qquad \text{padding share} = 1 - \frac{\sum_i \ell_i}{N \times T}$$

Worked example. Lengths 3, 7, 12, 2: $T = 12$, cells $= 48$, real tokens $= 24$, so **50% of the batch means nothing**. In Lab 7 a random batch of 8 real sentences was 52.1% padding.

T **comes from this batch, not from the dataset**, and changes every batch. That is what `collate_fn` is for, and a script that pads to a fixed global length has a bug.

By convention `<pad>` is id 0 and `<unk>` is id 1, with real tokens from 2. If you build the vocabulary without reserving those, your most frequent word gets id 0 and becomes indistinguishable from padding.

Part 2: Masking

A mask is a boolean tensor the same shape as the batch, marking which positions are real. It is `ids != PAD`. Without one, every layer downstream reads `PAD` as a word.

The exact error, which is the thing to remember

Because `padding_idx` pins the pad vector to zero, the unmasked mean is the sum of the real vectors divided by T, while the correct mean divides by ℓ:

$$\text{naive} = \frac{v}{T}, \qquad \text{masked} = \frac{v}{\ell} \qquad \Rightarrow \qquad \text{relative error} = 1 - \frac{\ell}{T}$$

Lab 7 measured this and the measured column matched the formula to the digit for all eight rows, with a correlation between length and error of exactly -1.

length	T	error
2	24	91.7%
12	24	50.0%
24	24	0.0%

Why a masking bug hides

It damages short examples in proportion to how short they are, and leaves the longest member of each batch untouched. So the validation average looks only mildly wrong. Wild behaviour on short inputs and clean behaviour on long ones points at masking before it points at the model.

The three places

where	what it looks like
pooling	divide by the real count, not T
the loss	`ignore_index=0`
attention	pad positions go to $-\infty$ before the softmax

Forgetting any one is silent. Nothing raises.

On the loss, Lab 7 compared two hypothetical models on a batch that was 52.1% padding. The one that got **every real word wrong** and every pad right scored an unmasked loss of **4.306**; the one that got every real word right scored **4.681**. Under an unmasked loss the useless model wins, which follows from more than half the target cells being `PAD`.

Part 3: Building the batch, and bucketing

A `Dataset` returns one example. Default collation stacks them and needs matching shapes, which ragged sequences do not have, so it raises.

`collate_fn` receives the list and returns whatever the model wants: pad to the longest member of *this* batch and hand back the mask alongside the ids.

Bucketing means sorting by length first, so batches become internally uniform and most padding disappears. Lab 7, on 2,048 sentences:

batch size	shuffled	sorted
8	60.2% padding	2.2%
32	73.5% padding	10.1%

In practice you shuffle, take a pool of a few hundred, sort inside the pool and batch from that, because sorting the whole dataset means every epoch sees the same batches.

The one place order is not yours to change: a time series. Split it on time, train on the past, validate on the future. A random split lets the model see tomorrow while it learns yesterday.

Part 4: 1D convolution

`nn.Conv1d` takes (N, C, T), note the order, and **accepts any T with the same weights**. So variable length was never the real objection to using convolution on text. Locality is.

A k-wide kernel widens the receptive field by $k-1$ per layer, so L stacked layers see $L(k-1)+1$ positions. With $k = 3$ that is $2L + 1$.

> **Likely question.** How many layers of kernel 3 before one position can see a token 100 away? You need 101 positions in view, so $2L + 1 \geq 101$ and $L \geq 50$. Attention needs one layer.

1D convolution is **current practice, not history**: a convolutional front end before a speech transformer, Conformer blocks, dilated causal convolutions for time series. Use it where the pattern is local.

Part 5: The recurrence, and why it breaks

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)\,, \qquad h_0 = \mathbf{0}$$

The same weights run at every step. That is **weight sharing**, the same idea as a kernel sliding over an image except over time, and it is what lets one model take any length.

It is also the problem: **the only route from the first token to the last is along the W_h arrows, one multiplication per step, with no other path.**

Reading the output

`nn.RNN` returns `(output, h_n)`. `nn.LSTM` returns `(output, (h_n, c_n))`. With `batch_first=True`, `output` is (N, T, H): the hidden state h_t at every step. The cell state c_t is not in `output`. Which part you read makes it a different task. **With padding, `out[:, -1]` is the state after reading pad tokens**, not after reading the sentence.

Generation and teacher forcing

Predict one token, append it, run again: one forward pass per token. Training does not work that way, because feeding the model its own early predictions compounds errors, so you feed the true previous token instead. That is **teacher forcing**, and it is why training is parallel over positions while generation is not.

Failure one: the gradient does not reach back

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

A product of $T - t$ terms. Smaller than 1 and it goes to zero. **No choice of weights makes a product of many terms behave like a sum.** Measured: the vanilla RNN keeps **0.53** of the gradient per step, a usable reach of about 20 steps.

Exploding is the easy half, you clip the norm. Vanishing is the hard half, because there is nothing to clip.

Say the cost precisely: it is not that the model is wrong about long sequences, **it is that it cannot learn anything that depends on one**, because the signal that would teach it never arrives.

Failure two: one step at a time

h_t needs h_{t-1}, so the steps cannot run together, no matter the hardware. Measured: an LSTM at **16.00 ms** against attention's **7.63 ms** on 1,024 tokens, and attention does *more* total arithmetic.

What the gates compute

c_t is the cell state, the memory. h_t is the hidden state, what the cell exposes. x_t is the input at this step.

Every circle reads h_{t-1} and x_t. It concatenates them, multiplies by its own weight matrix, adds its own bias, and squashes:

$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\
\tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c)
\end{aligned}
$$

f_t is the forget gate, i_t the input gate, and o_t the output gate. All three lie in $(0, 1)$. $\tilde{c}_t$ is the candidate. It uses tanh, so it lies in $(-1, 1)$.

$$
c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad h_t = o_t \odot \tanh(c_t)
$$

With f_t near 1, $c_t \approx c_{t-1}$, so the gradient along that track is multiplied by roughly 1 instead of a weight matrix. That is the residual connection's trick, twenty years earlier.

	fixed?
gradient reach	partly: 28 steps against 20, still exponential
one step per token	not at all

Part 6: Query, key and value

This is the one new idea in the transformer, and the most likely thing to be asked about. Every token produces three vectors from its own embedding, each by its own learned linear layer:

- **query** q: what this token is looking for
- **key** k: what this token offers to anyone looking
- **value** v: what this token contributes if attended to

$$
\mathbf{Q} = \mathbf{X}\mathbf{W}_\mathbf{Q}, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_\mathbf{K}, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_\mathbf{V}
$$

A token's attention to another is its query dotted with that token's key. Those scores become weights, and the output is the weighted sum of the values.

The library analogy: the query is your request, the keys are the spine labels, the values are the books.

In **self-attention** all three come from the same sequence. In **cross-attention** the queries come from the decoder and the keys and values come from the encoder.

Part 7: Scaled dot product attention

$$
\mathbf{A}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = S\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}
$$

Four steps: score every pair, divide by $\sqrt{d_k}$, softmax each row, multiply by the values.

Why the square root

A dot product of two d_k-dimensional vectors is a sum of d_k terms, so its spread grows like $\sqrt{d_k}$. Measured at $d_k = 512$: the unscaled softmax puts **91%** of its weight on one token, against **3%** when scaled.

Saturation is not the real cost. A saturated softmax has almost no gradient, so the layer stops learning: the scaled version delivers **6 times** more gradient to its logits.

The attention matrix

Sequence by sequence. Entry (i, j) is how much token i attends to token j, and **every row sums to 1**. It is $n \times n$, which is where the quadratic cost comes from, and it is recomputed from the content at every layer.

The output of an attention layer is the same shape as its input, (N, T, d).

Part 8: Multi-head attention

One attention layer produces **one** weight matrix, so it expresses one notion of relatedness per layer. Language has several at once.

Multi-head attention runs h attention operations in parallel on different learned projections of the same input, then concatenates.

Heads are free. With $d = 768$ and $h = 12$, each head works in $768/12 = 64$ dimensions, so the four projection matrices cost **2,359,296** parameters either way. You get 12 relationships for the price of one.

And they genuinely differ. Measured on a real DistilBERT layer: one head put **62%** of its weight on the previous token, another put **88%** on the separator.

Part 9: Position

Attention is order-blind. Every output is a weighted sum over the whole sequence and nothing in it depends on where a token sits, so shuffling the input shuffles the output with identical values. "The dog bit the man" and "the man bit the dog" are the same bag of tokens.

Positional encoding adds a vector to each token's embedding that depends only on its position.

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \qquad P(k, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

Each dimension is a wave of a different wavelength, so every position gets a distinct pattern. Measured: similarity between two encodings depends on **the gap**, not the place, with neighbouring positions at 0.97 and positions 20 apart at 0.61 from anywhere in the sequence.

Learned positional encoding is the alternative: one row per position, trained like any parameter, which is what BERT, GPT and ViT do. The trade is extrapolation: a learned table has a fixed number of rows, and **position 2,049 of a 2,048 row table does not exist**.

Part 10: The two masks

A padding mask stops attention from reading pad tokens. Needed in both the encoder and the decoder.

A causal mask stops a position attending to anything after it. Only in the decoder, and only because of generation: at inference the later tokens do not exist yet, so training must not let the model see them.

Both are implemented the same way: **add $-\infty$ before the softmax**, because $e^{-\infty} = 0$.

Measured on a real attention matrix: after the causal mask the first row is **1.00** on itself, because the first token has nothing before it, and every row still sums to **1.00**, because the softmax runs after the mask.

Part 11: The block

Attention is one sublayer. A block has two, each wrapped the same way:

$$\mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{Attention}(\mathbf{x})), \qquad \mathbf{x} \leftarrow \text{LayerNorm}(\mathbf{x} + \text{FFN}(\mathbf{x}))$$

- **The residual connection** is Week 6's, for the same reason: a path with derivative 1 so you can stack many blocks.
- **The feed forward network** is a two-layer MLP applied to each position independently, widening to $4d$ and back. Attention mixes across positions; the FFN is where per-position computation happens, and it is the **bigger half** of a block's parameters.
- **Layer normalization** is the part that is not Week 6's.

LayerNorm versus BatchNorm

LayerNorm normalizes each token by that token's own mean and standard deviation. **BatchNorm** normalizes it by the mean and standard deviation of the whole batch, and the padding zeros are part of that batch.

LayerNorm of a token is not a function of the padding or the other tokens. BatchNorm is. Hold one token fixed and replace other rows with zeros:

	how far the output moves as padding is added
BatchNorm, down the batch	**2.20**
LayerNorm, across the token	**0.00**

0.00 means the same output each time, because those rows are not inputs to the function. The output is still a different number from the input.

---

architecture	sees	example	good at

Part 12: Encoder, decoder, or both

architecture	sees	example	good at
encoder only	the whole input at once	BERT	classification, NER, extractive QA
decoder only	everything before this token	GPT	generating text
encoder-decoder	input fully, output causally	T5	translation, summarization

Bidirectional context is for understanding; causal masking is for generating. You cannot generate with bidirectional attention, because the thing you would attend to does not exist yet.

How to prepare

1. **Be able to define a query, a key and a value in one sentence each**, and say what the attention weight is. If you learn one thing, learn this.
2. **Know what each mask is for** and which half of the architecture it lives in.
3. **Be able to say why the $\sqrt{d_k}$ is there**, and that the real reason is gradient, not just saturation.
4. **Be able to do the padding arithmetic cold**: given four lengths, give T, the cell count and the padding share. And apply $1 - \ell/T$.
5. **Know the two failures of recurrence** and which one gating helped.
6. **Be able to say why LayerNorm and not BatchNorm** in one sentence: LayerNorm of a token is not a function of the padding or the other tokens.
7. **From Lab 8**, say what the target at each position is, why a causal mask's rows still sum to 1, and why generation takes one forward pass per new word while teacher forcing takes one pass for the whole sequence.
8. **Re-run Lab 7 and Lab 8 and read your own output.** Most of the numbers in this guide are ones you printed.

The formula sheet carries the attention equation, the recurrence, the product for backpropagation through time, the LSTM update, the padding formulas, the $1 - \ell/T$ identity, the sinusoidal encoding and the receptive field. It deliberately does not say what any of them are *for*. That part is the quiz.